UNIVERSIDAD TECNOLÓGICA DE CHIHUAHUA

Entornos Virtuales y Negocios Digitales

Programación de Aplicaciones Web Progresivas

Reporte de práctica: requisitos para conectar una aplicación web con una API y prepararla como PWA

PRESENTA:

Ángel Iván López Delgado

Docente: Juan Carlos Bustamante Lozano

Chihuahua, Chih., a 18 de septiembre de 2026

1. Introducción

Convertir un sitio web en una aplicación web progresiva no exige rehacer el producto. Exige cumplir un conjunto acotado de requisitos y agregar unas cuantas piezas de configuración. Lo mismo ocurre con la conexión a una API: más que escribir código, lo que determina el resultado es tener claro qué se necesita antes de empezar.

El presente reporte documenta la práctica desde ese ángulo: **qué se ocupa** para lograr cada objetivo, para qué sirve cada elemento y qué criterio se aplicó al decidirlo. No se incluye el detalle de implementación, sino los requisitos, las piezas involucradas, el orden de trabajo y las verificaciones que confirman que quedó bien hecho.

2. Objetivo

Identificar y describir los elementos necesarios para (a) comunicar una aplicación web con una API y (b) dejar esa aplicación preparada para funcionar como aplicación web progresiva, señalando la función de cada elemento y la manera de comprobar que cumple su propósito.

3. Entorno de la práctica

Elemento	Detalle
Equipo	Computadora con Node.js instalado
Marco de desarrollo	Next.js (React), el mismo del proyecto integrador
Navegador	Google Chrome, con sus herramientas de desarrollo
Herramienta de auditoría	Lighthouse, integrada en el navegador
Servicio de datos	Una API que entregue información en formato JSON
Alojamiento	Un servidor con certificado de seguridad (HTTPS)

4. Parte I. Qué se necesita para comunicar una aplicación web con una API

Una API es el punto por el que un servicio entrega o recibe información. La aplicación no guarda los datos: los pide. Para que esa comunicación funcione se ocupan seis elementos.

4.1 La documentación del servicio

Es el requisito previo a todo lo demás. De ella se obtienen cuatro datos indispensables: la dirección del servicio, qué información se puede pedir, en qué formato llega la respuesta y si hace falta una credencial para acceder. El error más común de la práctica es programar suponiendo el formato de la respuesta en vez de confirmarlo. Antes de integrar nada se probó cada consulta por separado, para ver la respuesta real.

4.2 Una credencial de acceso y un lugar seguro donde guardarla

La mayoría de los servicios exigen una clave que identifica a quien consulta. Esa clave **no se escribe dentro del código**, porque el código del lado del navegador puede leerlo cualquier usuario. Se guarda en un archivo de configuración aparte, que no se sube al repositorio, y se distingue explícitamente entre los datos que sí pueden ser públicos (la dirección del servicio) y los que deben permanecer del lado del servidor (la clave).

4.3 Un punto único donde ocurran las llamadas

En lugar de repartir consultas por toda la aplicación, se concentran en un solo módulo. La razón es de mantenimiento: si el servicio cambia de dirección, exige una cabecera nueva o empieza a fallar, se corrige en un solo lugar. Ahí mismo se define qué hacer cuando el servicio responde con error y cuánto tiempo esperar antes de darlo por caído.

4.4 Un intermediario en el servidor cuando la clave es privada

Cuando la credencial no puede exponerse, la aplicación no llama al servicio desde el navegador. Llama a una ruta propia y esa ruta, que corre en el servidor, es la que reenvía la consulta ya autenticada. Con esto se logran dos cosas: la credencial nunca sale del servidor y se evita el bloqueo por política de origen cruzado (CORS), que es el mecanismo por el cual un navegador impide que una página consulte un servicio que no la autorizó expresamente.

4.5 Un manejo explícito de los tres estados posibles

Toda consulta tiene tres desenlaces y la interfaz debe contemplar los tres, no solo el favorable:

Estado	Qué debe mostrar la aplicación
Esperando respuesta	Un indicador de carga, para que el usuario sepa que el sistema está trabajando
Error	Un mensaje comprensible y, de ser posible, la opción de reintentar
Respuesta vacía	Un aviso de que no hay información, distinto del mensaje de error

Omitir estos estados produce pantallas en blanco que el usuario interpreta como una falla del sistema.

4.6 Una verificación antes de dar por terminado

Se comprobó en las herramientas del navegador que la consulta saliera con las cabeceras correctas, que el servicio respondiera con código de éxito, que la información recibida coincidiera con la documentación y que la interfaz reaccionara adecuadamente al forzar un error.

5. Parte II. Qué se necesita para que la aplicación sea una PWA

Para que un navegador reconozca una aplicación web como instalable y pueda funcionar sin conexión se ocupan cinco elementos. Ninguno es opcional.

5.1 Un sitio servido por HTTPS

Es el requisito de entrada. El componente que permite el funcionamiento sin conexión solo se activa en sitios con certificado de seguridad. La razón es sensata: se trata de un componente capaz de intervenir todo el tráfico de la aplicación, y permitir eso sobre una conexión sin cifrar sería un riesgo. Durante el desarrollo se trabaja en la computadora local, que el navegador considera segura por excepción, pero la prueba definitiva se hace sobre el sitio ya publicado.

5.2 Un juego de iconos

El sistema operativo necesita una imagen para representar la aplicación una vez instalada. Se ocupan como mínimo dos tamaños, uno mediano y uno grande, y conviene una tercera versión con el logotipo centrado y con margen, para que los teléfonos que recortan los iconos a su propia forma no lo mutilen.

5.3 El manifiesto de la aplicación

Es un archivo de configuración que presenta la aplicación ante el sistema operativo. Sin él, el navegador no ofrece la opción de instalar. Contiene:

Dato	Para qué sirve
Nombre y nombre corto	Cómo se llama la aplicación y cómo se muestra bajo el icono
Iconos	Las imágenes descritas en el punto anterior
Página de inicio	Qué pantalla abre la aplicación al iniciarse
Modo de presentación	Si abre en ventana propia o dentro del navegador con su barra de direcciones
Colores de tema y fondo	La apariencia de la barra del sistema y de la pantalla de arranque
Orientación	Si se fija en vertical, horizontal o se adapta

El manifiesto debe además enlazarse desde la aplicación, y en el caso de los dispositivos de Apple se agregan algunos datos adicionales, porque su navegador no interpreta todos los campos del estándar.

5.4 El service worker

Es la pieza central del modelo. Se trata de un archivo que el navegador ejecuta en segundo plano, aparte de la página, y que funciona como intermediario entre la aplicación y la red: ve pasar cada petición y decide si la manda al servidor o la responde con una copia que ya tenía guardada. Es lo que permite que la aplicación siga funcionando sin conexión, y también lo que habilita las notificaciones y la sincronización en segundo plano.

Su instalación implica tres momentos: al instalarse guarda los archivos mínimos para que la aplicación arranque; al activarse borra las copias de versiones anteriores; y de ahí en adelante atiende cada petición según la regla que se le haya definido.

5.5 Una estrategia de caché decidida a conciencia

Este es el punto que realmente exige criterio, y donde se concentran las fallas difíciles de detectar. No todo el contenido se debe guardar igual:

Tipo de contenido	Regla aplicada	Por qué
Datos que vienen de la API	Primero la red; la copia guardada solo si no hay conexión	La información cambia, se prefiere el dato fresco pero sin dejar la pantalla vacía cuando falla la red
Iconos, tipografías y recursos fijos	Primero la copia guardada	No cambian entre versiones, servirlos localmente es inmediato
Página de respaldo sin conexión	Siempre guardada desde la instalación	Es lo que se muestra cuando el usuario navega a una sección que no está disponible

Una estrategia mal planteada no produce ningún mensaje de error: simplemente entrega contenido viejo, y por eso es tan difícil de diagnosticar.

5.6 Cómo se comprueba que quedó bien

La verificación se hizo desde las herramientas de desarrollo del navegador, en este orden:

#	Comprobación	Resultado esperado
1	Revisar el manifiesto	Carga sin advertencias y los iconos se muestran
2	Revisar el service worker	Aparece activo y en ejecución
3	Revisar el almacenamiento	Existe la caché con los archivos precargados
4	Activar el modo sin conexión y recargar	La aplicación sigue respondiendo
5	Ejecutar la auditoría de Lighthouse	Confirma que la aplicación es instalable
6	Instalarla realmente	Abre en ventana propia, sin barra del navegador

6. Resumen de lo que se ocupa

Objetivo	Lo que se necesita
Conectar con una API	Documentación del servicio - credencial guardada fuera del código - un punto único de consulta - un intermediario en el servidor si la clave es privada - manejo de carga, error y vacío - verificación en el navegador
Preparar la aplicación como PWA	Sitio con HTTPS - iconos en los tamaños requeridos - manifiesto - service worker - estrategia de caché por tipo de contenido - página de respaldo sin conexión

7. Problemas encontrados y su solución

#	Problema	Causa	Solución
1	El navegador no ofrecía instalar la aplicación	Faltaba uno de los tamaños de icono obligatorios	Se completó el juego de iconos en el manifiesto
2	Los cambios hechos no se reflejaban al recargar	La versión anterior del service worker seguía sirviendo contenido guardado	Se numeró la versión de la caché y se forzó la actualización
3	El servicio respondía bien pero el navegador bloqueaba la información	El servicio no autorizaba el origen de la aplicación (CORS)	Se pasó la consulta por el servidor de la propia aplicación

4	Sin conexión aparecía pantalla en blanco	No existía una página de respaldo	Se agregó y se incluyó entre los archivos precargados
5	El icono se veía recortado en Android	Se usaba el icono normal como adaptativo	Se agregó la versión con margen de seguridad

8. Conclusiones

La práctica confirma que preparar una aplicación como PWA es un trabajo de configuración con requisitos claros -seguridad en el sitio, iconos, manifiesto y service worker- y no una reescritura del producto. Lo verdaderamente delicado no es activar el service worker, sino **decidir qué se guarda y por cuánto tiempo**: ahí se juega que la aplicación se sienta rápida o que muestre información vieja sin avisar.

En la conexión con la API, lo determinante tampoco fue la programación, sino el orden: confirmar el contrato del servicio antes de integrarlo, mantener las credenciales fuera del alcance del navegador y contemplar desde el principio que la consulta puede tardar, fallar o no traer nada. Ambos frentes se complementan, porque es la combinación de datos remotos con una política de caché explícita lo que permite que la aplicación siga siendo útil cuando la red no responde.

9. Referencias

- MDN Web Docs. *Progressive web apps* y *Service Worker API*. developer.mozilla.org
- Google Developers. *Learn PWA*. web.dev/learn/pwa
- W3C. *Web Application Manifest*. w3.org/TR/appmanifest
- Vercel. *Next.js Documentation: Progressive Web Applications*. nextjs.org/docs
- Google Chrome Developers. *Caching strategies*. developer.chrome.com